

PMSCS 622P

Programming Languages

Lecture 06: Memory Management

Mr. Md. Masum Bhuiyan

Department of Computer Science and Engineering
Jahangirnagar University

Part 1: What is Memory?

Before we code, we must understand the hardware.

RAM (Random Access Memory)

- It is a massive array of "Lockers".
- Each locker has a unique number (**Address**).
- Each locker holds a small value (1 Byte).

When you run a program, the OS gives it a slice of this RAM.

0x500	Empty
0x400	Empty
0x300	Empty
0x200	Empty
0x100	Empty

Variables are Nicknames

Computers speak **Addresses** (e.g., 0x7FFF). Humans speak **Names** (e.g., score).

The Compiler's Job

The compiler maintains a table connecting your names to the machine's addresses.

```
int score = 95;
```



*"Put value **95** into Address **0x402**."*

How Programs Organize Memory

Your program's memory isn't just one big bucket. It is divided into zones.

- **Code Segment:** Where your compiled instructions live (Read Only).
- **Data Segment:** Global variables (available everywhere).
- **Stack:** Temporary variables (Function calls).
- **Heap:** Long-term memory (Objects/Data structures).

The Two Giants: Stack vs. Heap

We focus on the two areas where *your* data lives.

The Stack (Ordered)

- Think: **Stack of Plates**.
- Very fast.
- Strictly LIFO (Last In, First Out).
- Small size (MBs).

The Heap (Chaotic)

- Think: **Open Floor**.
- Slower to access.
- Random access (put things anywhere).
- Very Large (GBs).

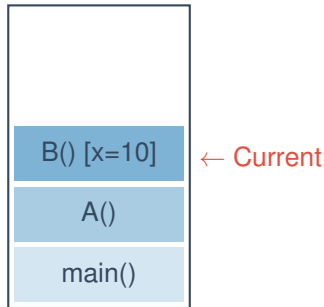
Deep Dive: The Stack

Purpose: Keeps track of function calls.

- When you call a function, a "**Stack Frame**" is pushed.
- A Frame contains:
 - 1 Parameters (Arguments passed to function).
 - 2 Local Variables (Created inside function).
 - 3 Return Address (Where to go back to).
- When function ends, the Frame is **Popped** (deleted instantly).

Visualizing Stack Execution

```
void B() {  
    int x = 10;  
}  
  
void A() {  
    B();  
}  
  
int main() {  
    A();  
    return 0;  
}
```



Stack Properties

- **Allocation is Automatic:** You don't ask for memory. You just declare `int x;`
- **Deallocation is Automatic:** When the closing curly brace `}` is reached, memory is gone.
- **Scope Rule:** You cannot access a variable in `'main()'` from inside `'B()'`. Why? Because they are in different stack frames (different "rooms").

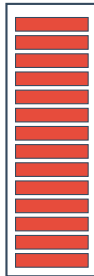
The Danger: Stack Overflow

The Stack has a limit. If you keep pushing without popping, you crash.

```
// Infinite Recursion
void runForever() {
    runForever();
}
```

This fills the stack in milliseconds.

OVERFLOW!



Deep Dive: The Heap

Purpose: Storing data that needs to live longer than a single function.

- Also called **Dynamic Memory**.
- **Explicit:** YOU must request it.
- **Persistent:** It stays until you delete it (or GC cleans it).
- **Flexible Size:** Need an array of 1 million items? Use the Heap.

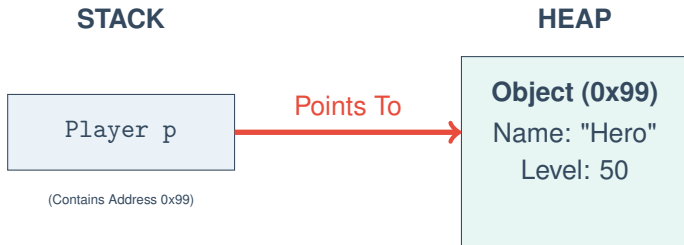
The Bridge: Pointers/References

We cannot "hold" the Heap. We can only hold the **address** of something in the Heap.

- The variable lives on the **Stack**.
- The data lives on the **Heap**.
- The Stack variable acts as a **Remote Control**.

```
// 'p' is the remote control (Stack)
// 'new Player()' is the TV (Heap)
Player p = new Player();
```

Visualizing the Bridge



Manual Management (C++)

C++ gives you raw power. You are the manager of the warehouse.

- 1 **Allocate:** `new int(5)` → Finds space in Heap, returns address.
- 2 **Use:** Read/Write data.
- 3 **Deallocate:** `delete ptr` → Marks space as free.

Risk: If you forget step 3, you run out of space.

Syntax: Pointers in C++

Many beginners struggle here. Let's simplify.

The Symbols

- * (in declaration): "I am a pointer."
- new: "Create on Heap."
- * (in usage): "Go to the address and get value."
- &: "Tell me your address."

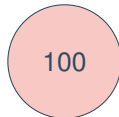
```
int* ptr = new int; // Create a pointer to a new integer
*ptr = 10;          // Go to that address, write 10
delete ptr;         // Free the memory
```

Memory Leak: The Silent Killer

```
void createLeak() {  
    // 1. Allocate memory  
    int* x = new int(100);  
  
    // 2. Function ends  
    // 'x' (stack) is destroyed.  
    // Memory (heap) is NOT freed.  
}
```

x (popped) - - → ?

Orphaned Data



If you call this function 1,000,000 times, your RAM is full. Crash.

Dangling Pointers

Problem: You deleted the house, but you kept the key.

```
int* p = new int(10);  
delete p;  // Memory is freed. OS takes it back.  
  
// ... later in code ...  
  
*p = 20;    // DANGER! You are writing to memory  
            // that might now belong to Chrome or Windows.
```

Solution: Always set `p = nullptr`; after deleting.

Why trust humans? Humans make mistakes.

Garbage Collection (GC)

An automated janitor service.

- You create objects freely (`new`).
- You never delete.
- The GC runs in the background, finds unused objects, and removes them.

How does GC know what is "Garbage"?

The definition of Garbage:

"An object that can no longer be reached by the running program."

If you lose the "Remote Control" (Reference), you can never use the TV (Object) again. It becomes garbage.

Algorithm 1: Reference Counting (Python)

Idea: Keep a count of how many pointers point to an object.

- `a = new Object()` → Count = 1
- `b = a` → Count = 2
- `a = null` → Count = 1
- `b = null` → Count = 0 (**DELETE NOW!**)

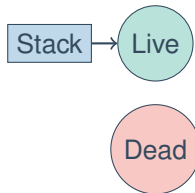
Mainly used in Python. Simple, but fails with circular references (A points to B, B points to A).

Algorithm 2: Mark and Sweep (Java)

Idea: Occasionally stop the world and trace everything.

Step 1: Mark Start from the Stack ("Roots").
Follow every pointer. Mark every object found as "Alive".

Step 2: Sweep Scan the Heap. Any object NOT marked is dead. Delete it.



The Cost of Magic

GC makes life easy, but it is not free.

- **Performance Overhead:** The CPU has to spend time counting/sweeping instead of running your app.
- **Unpredictable Pauses:** Sometimes the program "freezes" for 10ms while GC runs.
- **Memory Footprint:** GC languages often use more RAM than C++.

Comparison Summary

Feature	C++ (Manual)	Java/Python (Auto)
Heap Alloc	new	new (or implied)
Heap Free	delete	<i>Automatic (GC)</i>
Pointer Risk	High (Dangling)	Low (NullPointer only)
Leak Risk	High	Low (but possible)
Speed	Maximum	Good (with overhead)

Class Quiz 1: Where does it go?

I will show a line of code. You say: **STACK** or **HEAP**.

- 1 `int age = 25;`
- 2 `String s = new String("Hello");`
- 3 `double[] prices = new double[100];`
- 4 The variable `prices` itself?

Class Quiz 1: Where does it go?

I will show a line of code. You say: **STACK** or **HEAP**.

- 1 `int age = 25;` → **STACK**
- 2 `String s = new String("Hello");`
- 3 `double[] prices = new double[100];`
- 4 The variable `prices` itself?

Class Quiz 1: Where does it go?

I will show a line of code. You say: **STACK** or **HEAP**.

- 1 `int age = 25;` → **STACK**
- 2 `String s = new String("Hello");` → **HEAP** (Object)
- 3 `double[] prices = new double[100];`
- 4 The variable `prices` itself?

Class Quiz 1: Where does it go?

I will show a line of code. You say: **STACK** or **HEAP**.

- 1 `int age = 25;` → **STACK**
- 2 `String s = new String("Hello");` → **HEAP** (Object)
- 3 `double[] prices = new double[100];` → **HEAP**
- 4 The variable `prices` itself?

Class Quiz 1: Where does it go?

I will show a line of code. You say: **STACK** or **HEAP**.

- 1 `int age = 25;` → **STACK**
- 2 `String s = new String("Hello");` → **HEAP** (Object)
- 3 `double[] prices = new double[100];` → **HEAP**
- 4 The variable `prices` itself? → **STACK** (Pointer)

Class Quiz 2: Spot the Bug (C++)

```
int* getNumber() {  
    int x = 50;  
    return &x; // Returning address of x  
}  
  
void main() {  
    int* p = getNumber();  
    cout << *p;  
}
```

Class Quiz 2: Spot the Bug (C++)

```
int* getNumber() {  
    int x = 50;  
    return &x; // Returning address of x  
}  
  
void main() {  
    int* p = getNumber();  
    cout << *p;  
}
```

ERROR: `x` is on the Stack. When `getNumber` ends, `x` is destroyed. `p` is now pointing to a ghost.

Final Takeaway

- **Stack** is for temporary execution context (Functions).
- **Heap** is for long-term data storage (Objects).
- In **C++**, you manage the Heap. Be careful.
- In **Java/Python**, the GC manages the Heap. Be grateful.

Next Step: Lecture 07 - Class and Objects.